Copyright All rights reserved

07. 제어문:반복문
(Control Flow – Loop)

Heesung Oh
https://github.com/3dapi

7.1 for 문 기본 구조

Heesung Oh
Copyright All rights reserved

- 반복문: for, while, do-while

- for statement

```
for (초기식; 조건식; 증감식)
{
    반복 실행할 문장;
}
```

초기화 → 조건 확인
→ 반복 문장 실행 → 증감

- 초기식
  - 반복 변수 초기화. 반복 시작 시 1회 실행
- 조건식
  - 반복 지속 여부 판단. 참 → 반복 실행, 거짓 → 반복 종료
- 증감식
  - 반복마다 변수 변화. 다음 반복 준비

7.1.1 for: 초기식 생략

Heesung Oh
Copyright All rights reserved

- 변수 재사용 목적 → 초기식 생략

```
#include <stdio.h>

int main(void)
{
    int jobCount = 4;     // 실행해야 할 작업 개수(상황에 따라 결정됨)
    int playCount = 0;    // 반복문 앞에서 선언 및 초기화

    // 초기식 생략
    for (; playCount < jobCount; ++playCount)
    {
        // 어떤 작업 수행
    }

    printf("반복 종료 후 playCount = %d 회 실행되었습니다.\n"
             , playCount);
    return 0;
}
```

7.1.2 for: 증감식 생략

Heesung Oh
Copyright All rights reserved

- 반복 내부에서 직접 증가 또는 감소 수행

```
for (int i = 0; i < 10; )
{
    printf("%d\n", i);

    i += 2;     // 증감식 대신 반복문 내부에서 증가
}
```

7.1.3 for: 초기식, 증감식 생략, 조건식만 구현

Heesung Oh
Copyright All rights reserved

- == while(조건식)

```
int i = 0;

for (; i < 10; )     // 초기식 없음, 증감식 없음
{
    printf("%d\n", i);

    ++i;             // 반복문 내부에서 증가
}
```

7.1.4 for: 모두 생략 (무한 루프)

Heesung Oh
Copyright All rights reserved

- 무한 루프 표현
- == while(1)

```
for (;;)
{
    // 반복 실행할 문장
    if (/* 특정 조건 */)
        break;      // 직접 탈출
}
```

7.2.1 for 문 기본 예제1 (Simple for Loop)

Heesung Oh
Copyright All rights reserved

- 〈예: [0, 100] 정수 출력〉

```
#include <stdio.h>

int main(void)
{
    for (int i = 0; i <= 10; ++i)
    {
        printf("%d\n", i);
    }

    return 0;
}
```

7.2.2 for 문 기본 예제: 합계 구하기 (Summation)

Heesung Oh
Copyright All rights reserved

- 〈예: [0, 1000] 합 구하기〉

```
#include <stdio.h>

int main(void)
{
    int sum = 0;
    int nBegin = 0, nEnd = 1000;

    for (int i = nBegin; i <= nEnd; ++i)
    {
        sum += i;
    }

    printf("%d ~ %d 까지의 합: %d\n", nBegin, nEnd, sum);
    return 0;
}
```

7.2.3 for 문 기본 예제: 증감 식

Heesung Oh
Copyright All rights reserved

- 예: [0, 100] 의 정수 중 홀수만 합계 구하기

```
int sum = 0;
// 홀수만 순차적으로 증가
for (int i = 1; i <= 100; i += 2)
{
    sum += i;
}
```

- 예: [0, 1000] 의 정수 중 7의 배수 합계 구하기

```
int sum = 0;
// 7씩 증가 → 7의 배수만 처리
for (int i = 0; i <= 1000; i += 7)
{
    sum += i;
}
```

7.2.4 for 문 기본 예제: 숫자 자릿수 구하기

Heesung Oh
Copyright All rights reserved

- n % 10 → 마지막 자릿수, n /= 10 → 자릿수 감소
- 뒤에서부터 출력, 전체 자릿수 계산
- 초기식 생략, 감산 반복 구조

```c
#include <stdio.h>

int main(void)
{
    int n = 12345678;   // 상황에 따라 주어진 숫자
    int digit = 0;      // 자릿수 저장 변수

    for(; n > 0; n /= 10)       // 초기식 생략, n을 계속 나누면서 감소
    {
        printf("%d ", n % 10);  // 가장 뒤 자릿수 출력
        ++digit;                // 자릿수 증가
    }

    printf("\nTotal digits: %d\n", digit);  // 총 자리수 출력
    return 0;
}
```

7.2.5 for 문의 비 권장 형태

Heesung Oh
Copyright All rights reserved

- 문법 유연
  - 다양한 형태 작성 가능

- 문제점
  - 가독성 저하
  - 디버깅 어려움
  - 협업 시 오해 발생

7.2.5 for 문의 비 권장 형태

Heesung Oh
Copyright All rights reserved

- **초기식·증감식에 실행 코드**
  - 설정 / 검사 영역에 실행 코드 혼합
  - 가독성 저하
  - 동작 이해 어려움

```
for (printf("초기식 실행\n");
     i < 10;
     printf("증감식 값: %d\n", i), i += 2)
```

- **초기식 다중 변수**
  - 여러 변수 동시 선언 가능
  - 반복 목적 흐림
  - 가독성 저하

```
for (int i=0, j=0, k=0, l=0, m=0, n=0;
     i < 1;
     ++i, ++j, ++k, ++l, ++m, ++n)
```

- **중괄호 생략**
  - 빈 문장 반복 구조
  - 의도 파악 어려움
  - 수정 시 오류 위험

```
// 본문이 없는 완전한 형태 (비추천)
for (int i = 1; i < 10; ++i, n += i);
```

7.2.5 for 문의 비 권장 형태

Heesung Oh
Copyright All rights reserved

- 한 줄 for 문
  - 가독성 저하
  - 디버깅 어려움
  - 브레이크포인트 설정 불편

```
for (int i = 0; 0; ++i) printf("\n");
```

```
// 가독성과 디버깅을 고려한 방식
for (int i = 0; 0; ++i)
    printf("\n");

// 또는 블록을 명시적으로 사용
for (int i = 0; 0; ++i)
{
    printf("\n");
}
```

- 한 줄 블록
  - 컴팩트 코드
  - 의도 오해 가능
  - 유지보수 불리

```
for (int i = 0; 0; ++i) { printf("\n"); }   // 비권장
```

7.3.1 다중 for 문

Heesung Oh
Copyright All rights reserved

- **중첩 for 문**
  - for 내부에 또 다른 for 포함 구조

```
#include <stdio.h>

int main(void)
{
    printf(" 구구단 프로그램 \n");
    for(int table = 2; table <= 9; ++table)   // 바깥 반복문: 단(table)
    {
        printf("%d 단\n", table);             // 각 단 출력
        for(int num = 2; num <= 9; ++num)     // 안쪽 반복문: 곱하는 수(num)
        {
            int multi = table * num;          // 곱하기 결과
            printf("%d x %d = %d\n", table, num, multi);
        }
        printf("\n");                         // 각 단 출력 후 줄 구분
    }

    return 0;
}
```

Heesung Oh
Copyright All rights reserved

- C99 이전 표준
  - 레거시 대응

```c
#include <stdio.h>

int main(void)
{
    /* 사용할 변수는 미리 선언해 둔다.*/
    int table = 0;    /* 단(table) */
    int num = 0;      /* 곱하는 수(numb)*/
    int multi = 0;    /* 곱하기 결과 */

    printf(" 구구단 프로그램 \n");
    for(table = 2; table <= 9; ++table) /* 바깥 반복문: 단(table) */
    {
        printf("%d 단\n", table);       /* 각 단 출력 */
        for(num = 2; num <= 9; ++num)   /* 안쪽 반복문: 곱하는 수(num) */
        {
            multi = table * num;
            printf("%d x %d = %d\n", table, num, multi);
        }
        printf("\n");                   /* 각 단 출력 후 줄 구분 */
    }
    return 0;
}
```

7.3.3 패턴 출력: 별찍기

Heesung Oh
Copyright All rights reserved

- **왼쪽 정렬된 직각삼각형 별찍기**

```c
#include <stdio.h>

int main(void)
{
    int maxValStar = 4;
    for(int i = 1; i <= maxValStar; ++i)   // i는 줄(row)
    {
        for(int j = 1; j <= i; ++j)     // j는 각 줄의 별 개수(column)
        {
            printf("*");
        }
        printf("\n");                   // 한 줄 출력 후 줄바꿈
    }

    return 0;
}
```

7.3.3.2 오른쪽 정렬된 역삼각형 별찍기

Heesung Oh
Copyright All rights reserved

```
// 1. 공백 출력 과정
    공백의 개수 = (최대 별 개수) - (현재 출력할 별의 수)
    // 개행 없이 공백 출력
    for (s = 0; s < 공백의_개수; ++s)
        printf(" ");

// 2. 별을 출력 과정
    별의 개수 = 현재 출력할 별의 수
    for (j = 0; j < 별의_개수; ++j)
        printf("*");

// 3. 줄바꿈 출력
    printf("\n");
```

```c
#include <stdio.h>

int main(void)
{
    int maxStar = 4;                        // 최대 별 개수
    for(int i = maxStar; i >= 1; --i)       // i 는 출력할 별의 수
    {
        int numSpace = maxStar - i;         // 공백의 개수
        for(int s = 0; s < numSpace; ++s)   // 1. 공백 출력 과정
        {
            printf(" ");                    // 공백 출력
        }

        for(int j = 0; j < i; ++j)          // 2. 별 출력 과정: i 는 출력할 별의 수
        {
            printf("*");                    // 별 출력
        }
        printf("\n");                       // 3. 줄 바꿈 출력
    }

    return 0;
}
```

7.4 while 문 기본 구조

Heesung Oh
Copyright All rights reserved

- while 문
  - 조건이 참인 동안 반복 실행
  - 반복 횟수 미정 상황 적합
- 동작 방식
  - 조건 검사 → 실행 → 재검사
  - 조건이 처음부터 거짓 → 실행 안 함
- for vs while
  - for:횟수 기반 반복
  - while: 조건 기반 반복
- while 사용 예
  - 파일 입력
  - 네트워크 수신

7.4 while 문 기본 구조

Heesung Oh
Copyright All rights reserved

```
while (조건식)
{
    반복 실행할 문장;
}
```

```
for (; 조건식;)
{
    반복 실행할 문장;
}
```

```
// for 문을 while 문으로 변환한 형태
{
    초기식;                 // for 문의 초기식
    while (조건식)          // for 문의 조건식
    {
        반복 실행할 문장;
        증감식;             // for 문의 증감식
    }
}
```

7.5 while 문 예제

Heesung Oh
Copyright All rights reserved

```c
//  0~1000까지의 정수 중 7의 배수 합계 구하기
int sumSeven = 0;
// for 문에서 사용한 int i를 while에서만 접근하도록 블록으로 감쌈
{
    int i = 0;
    while(i <= 1000)
    {
        sumSeven += i;
        // 증감식
        i += 7;  // 7씩 증가 → 7의 배수만 처리
    }
}
printf("1~1000까지  7의 배수 합: %d\n", sumSeven);
```

```c
int n = 12345678;   // 상황에 따라 주어진 숫자
int digit = 0;      // 자릿수 저장 변수
while(n > 0)
{
    printf("%d ", n % 10);   // 가장 뒤 자릿수 출력
    ++digit;                 // 자릿수 증가
    n /= 10;                 // 증감식
}
printf("\nTotal digits: %d\n", digit);  // 총 자리수 출력
```

7.6 다중 while 문

Heesung Oh
Copyright All rights reserved

- 다중 while 문
  - while 내부에 또 다른 while 구조
  - 조건 기반 반복 중첩
- 활용
  - 입력 처리, 문자열 탐색, 파일 읽기
- 주의사항
  - 조건식 정확히 설정 ← 무한 루프 위험 존재
  - 반복 변수 반드시 변경, 종료 조건 명확히 정의
- 사용 기준
  - 반복 횟수 불명확 → while
  - 횟수 명확 → for

7.6.2 다중 while 문 예제: 행과 열 출력

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>

int main(void)
{
    int rows = 3;
    int cols = 5;
    int r = 1;
    while (r <= rows)
    {
        int c = 1;
        while (c <= cols)
        {
            printf("(%d, %d) ", r, c);
            c++;
        }
        printf("\n");
        r++;
    }
    return 0;
}
```

```
(1,1) (1,2) (1,3) (1,4) (1,5)
(2,1) (2,2) (2,3) (2,4) (2,5)
(3,1) (3,2) (3,3) (3,4) (3,5)
```

7.6.3 다중 while 문 예제: 9X9 table

Heesung Oh
Copyright All rights reserved

```
printf(" 구구단 프로그램 \n");
{

    int table = 2;
    while(table <= 9)                    // 바깥 반복문: 단(table)
    {
        printf("%d 단\n", table);        // 각 단 출력
        {
            int num = 2;
            while(num <= 9)              // 안쪽 반복문: 곱하는 수(num)
            {
                int multi = table * num;// 곱하기 결과
                printf("%d x %d = %d\n", table, num, multi);
                ++num;
            }
        }
        printf("\n");                    // 각 단 출력 후 줄 구분
        ++table;
    }
}
```

7.7 do - while 문

Heesung Oh
Copyright All rights reserved

- do - while 문
  - 조건과 관계없이 1회 실행, 조건이 참인 동안 반복 실행 (while)
- 동작 순서
  - 실행 → 조건 검사 → 반복 판단
- while과 차이
  - while: 조건 먼저 검사
  - do - while: 실행 후 조건 검사
- 주의
  - while 끝 세미콜론(;) 필수

```
do
{
    반복 실행할 문장;
} while (조건식);
```

7.7.2 do - while 문 기본 예제

Heesung Oh
Copyright All rights reserved

- **사용자 입력 확인**

```
#include <stdio.h>

int main(void)
{
    int value=-1;

    do
    {
        printf("0 이상의 정수를 입력하세요: ");
        scanf("%d", &value);
    } while (0 <= value);

    printf("입력한 값: %d\n", value);
    return 0;
}
```

7.7.3 do - while 문 기본 예제

Heesung Oh
Copyright All rights reserved

- **메뉴 처리**

```
#include <stdio.h>

int main(void)
{
    int choice= -999999;

    do
    {
        printf("\n=== 메뉴 ===\n");
        printf("1. 게임 시작\n");
        printf("2. 설정\n");
        printf("3. 종료\n");
        printf("번호를 선택하세요: ");
        scanf("%d", &choice);
    }
    while (choice != 3);

    printf("프로그램을 종료합니다.\n");
    return 0;
}
```

7.8.1 break

Heesung Oh
Copyright All rights reserved

- **break;**
  - 가장 가까운 반복문 또는 switch 즉시 종료. 조건과 무관하게 탈출
  - 사용 위치: switch, for, while, do – while

```
for (int i = 0; i < 100; ++i)
{
    if (i == 42)
        break;   // 발견 즉시 루프 종료
    printf("%d ", i);
}
```

```
int choice = 0;
do
{
    printf("1. 시작  2. 설정  3. 종료\n");
    scanf("%d", &choice);
    if (choice == 3)
        break;    // 메뉴 루프 종료
    printf("선택: %d\n", choice);
} while (1);
printf("프로그램을 종료합니다.\n");
```

```
for (int i = 0; i < 1024; ++i)
{
    for (int j = 0; j < 1024; ++j)
    {
        break;  // j 반복문만 종료됨
    }
}
```

7.8.2 continue

Heesung Oh
Copyright All rights reserved

- continue 문
  - 현재 반복 남은 문장 건너뜀
  - 다음 반복으로 이동
- 동작
  - for : 증감식으로 이동
  - while / do – while : 조건 검사로 이동

```
for (int i = 1; i <= 10; ++i)
{
    if (i % 2 == 0)
        continue;          // 짝수는 건너뛰기

    printf("%d ", i);
}
```

7.8.3 goto

Heesung Oh
Copyright All rights reserved

- goto 문
  - 지정한 레이블로 즉시 이동
  - 임의 위치 점프
  - 구조 흐름 붕괴 위험
- 사용
  - 제한적 사용 권장
  - 흐름 간결화

```
goto LABEL;

//...

LABEL:
```

7.8.3.1 goto 활용: 에러 처리 (Error Handling)

Heesung Oh
Copyright All rights reserved

```c
int main(void)
{
    while(1)
    {
        int choice = -999999;

        printf("\n=== 메뉴 ===\n");
        printf("1. 게임 시작\n");
        printf("2. 설정\n");
        printf("3. 종료\n");
        printf("번호를 선택하세요: ");
        scanf("%d", &choice);

        if (choice == 3)
            goto END_NORMAL;   // 정상 종료 지점으로 이동

        if (choice < 1 || choice > 3)
            goto END_ERROR;    // 비정상 종료 지점으로 이동

        // 1 또는 2를 선택한 경우 처리 코드가 들어갈 자리
        printf("선택: %d\n", choice);
    }

END_NORMAL:
    printf("프로그램을 종료합니다.\n");
    return 0;

END_ERROR:
    printf("잘못 선택된 메뉴입니다. 비정상 종료.\n");
    return 0;
}
```

7.8.3.2 goto 활용: 자원 정리 패턴

Heesung Oh
Copyright All rights reserved

- C 언어
  - 예외 처리 없음
  - 자동 자원 정리 없음
- goto 필요 상황
  - 여러 단계 초기화 과정
  - 중간 실패 가능
  - 공통 지점으로 이동, 정리
  - 중복 코드 제거
  - 흐름 단순화

```c
int loadData(const char* filename)
{
    FILE* fp     =  NULL;        // 파일 리소스 초기화
    char* buffer =  NULL;        // 메모리 리소스 초기화
    int   result = -1;           // -1 실패: 0: 성공

    fp = fopen(filename, "r");  // 파일 열기
    if(fp == NULL)
        goto CLEAN_UP;           // 파일 열기 실패

    buffer = (char*)malloc(1024); // 메모리 할당
    if(buffer == NULL)
        goto CLEAN_UP;           // 메모리 할당 실패

    // 여기서 파일 읽기, 처리 등 여러 작업 수행
    // ...

    result = 0;                  // return 값 성공으로 변경
CLEAN_UP:
    // 자원 해제
    if(fp != NULL)
    {
        fclose(fp);
    }
    if(buffer != NULL)
    {
        free(buffer);
    }
    return result;               // 결과 반환
}
```

7.8.3.3 goto vs do - while(0) 패턴

Heesung Oh
Copyright All rights reserved

- goto
  - 레이블로 직접 점프, 여러 종료 지점 구성 가능
  - 복잡한 분기: 자원 정리(cleanup)에 유리
- do - while(0)
  - do - while(0): 단일 블록 내부 제어
  - break로 조기 종료
  - 구조 유지: 가독성 우수
- 선택 기준
  - 복잡한 정리 / 다중 분기 → goto
  - 단순 흐름 제어 / 조기 종료 → do - while(0)

7.8.3.3 제어흐름 do - while(0) 패턴

Heesung Oh
Copyright All rights reserved

```c
do
{
    if (조건1)
        break;

    if (조건2)
        break;

    printf("여러 조건을 통과한"
           "경우에만 실행되는 코드\n");

} while (0);
```

```c
int loadData(const char* filename)
{
    FILE* fp     =  NULL;       // 파일 리소스 초기화
    char* buffer =  NULL;       // 메모리 리소스 초기화
    int   result = -1;          // -1 실패: 0: 성공

    do
    {
        fp = fopen(filename, "r");    // 파일 열기
        if (fp == NULL)
            break;              // 파일 열기 실패

        buffer = (char*)malloc(1024); // 메모리 할당
        if (buffer == NULL)
            break;              // 메모리 할당 실패

        // 여기서 파일 읽기, 처리 등 여러 작업 수행
        // ...
        //
        result = 0;             // return 값 성공으로 변경
    } while (0);

    // 자원 해제
    if (fp != NULL)
    {
        fclose(fp);
    }
    if (buffer != NULL)
    {
        free(buffer);
    }
    return result;              // 결과 반환
}
```

7.8.4 return

Heesung Oh
Copyright All rights reserved

- 현재 함수 즉시 종료
- 결과 값 호출자에게 전달

```
int divide(int a, int b)
{
    if (b == 0)
        return 0;// 잘못된 입력 → 조기 종료

    return a / b;
}
```

```
int findIndex(int searchRow, int searchCol)
{
    for (int i = 0; i < 100; ++i)
    {
        for (int j = 0; j < 100; ++j)
        {
            if (i == searchRow && j == searchCol)
                return i * 100 + j;      // 발견 즉시 함수 종료
        }
    }
    return -1;                           // 찾지 못한 경우
}
```

학습 정리

Heesung Oh
Copyright All rights reserved

- for 문
  - 초기식 → 조건식 → 증감식
  - 반복 횟수 명확한 경우 적합
  - 다중 for → 2차원 반복 구조
  - 가독성 저하 형태 사용 유의
- while 문
  - 조건 만족 동안 반복
  - 반복 횟수 불명확 상황 적합
  - 종료 조건 설계 중요
  - 무한 루프 주의
- do – while 문
  - 실행 → 조건 검사
  - 최소 1회 실행 보장
- 흐름 제어문
  - break : 반복문 / switch 종료
  - continue: 다음 반복으로 이동
  - goto:레이블로 점프 → 제한적 사용
  - return: 함수 종료 + 값 반환

Heesung Oh
Copyright All rights reserved

- 흐름 제어 패턴
  - goto
    - cleanup, 에러 처리
  - do - while(0)
    - 블록 단위 조기 종료

- 선택 기준
  - 횟수 기반 → for
  - 조건 기반 → while
  - 복잡 분기 → goto
  - 단순 흐름 → do - while(0)

Heesung Oh
Copyright All rights reserved

- 흐름 제어 패턴
  - goto
    - cleanup, 에러 처리
  - do - while(0)
    - 블록 단위 조기 종료

- 선택 기준
  - 횟수 기반 → for
  - 조건 기반 → while
  - 복잡 분기 → goto
  - 단순 흐름 → do - while(0)